

```

// C++ Program for Floyd Warshall Algorithm

#include <bits/stdc++.h>

using namespace std;

// Number of vertices in the graph
#define V 4

/* Define Infinite as a large enough
value.This value will be used for
vertices not connected to each other */
#define INF 99999

// A function to print the solution matrix
void printSolution(int dist[][V]);

// Solves the all-pairs shortest path
// problem using Floyd Warshall algorithm
void floydWarshall(int dist[][V])
{
    int i, j, k;

    /* Add all vertices one by one to
    the set of intermediate vertices.
    ---> Before start of an iteration,
    we have shortest distances between all
    pairs of vertices such that the
    shortest distances consider only the
    vertices in set {0, 1, 2, .. k-1} as
    intermediate vertices.
    ----> After the end of an iteration,

```

vertex no. k is added to the set of

intermediate vertices and the set becomes {0, 1, 2, ..

k} \*/

```
for (k = 0; k < V; k++) {
```

```
    // Pick all vertices as source one by one
```

```
    for (i = 0; i < V; i++) {
```

```
        // Pick all vertices as destination for the
```

```
        // above picked source
```

```
        for (j = 0; j < V; j++) {
```

```
            // If vertex k is on the shortest path from
```

```
            // i to j, then update the value of
```

```
            // dist[i][j]
```

```
            if (dist[i][j] > (dist[i][k] + dist[k][j]))
```

```
                && (dist[k][j] != INF
```

```
                && dist[i][k] != INF))
```

```
                dist[i][j] = dist[i][k] + dist[k][j];
```

```
        }
```

```
    }
```

```
}
```

```
// Print the shortest distance matrix
```

```
printSolution(dist);
```

```
}
```

```
/* A utility function to print solution */
```

```
void printSolution(int dist[][V])
```

```
{
```

```
    cout << "The following matrix shows the shortest "
```

```
            "distances"
```

```
            " between every pair of vertices \n";
```

```
    for (int i = 0; i < V; i++) {
```

```

        for (int j = 0; j < V; j++) {
            if (dist[i][j] == INF)
                cout << "INF"
                    << " ";
            else
                cout << dist[i][j] << " ";
        }
        cout << endl;
    }
}

```

// Driver's code

```
int main()
```

```
{
```

```
    /* Let us create the following weighted graph
```

```

                10
(0)----->(3)
    |      /\
5 |      |
    |      | 1
    \      |
(1)----->(2)
                3      */

```

```
int graph[V][V] = { { 0, 5, INF, 10 },
```

```
                { INF, 0, 3, INF },
```

```
                { INF, INF, 0, 1 },
```

```
                { INF, INF, INF, 0 } };
```

```
// Function call
```

```
floydWarshall(graph);
```

```
return 0;
```

```
}
```

```
// This code is contributed by Mythri J L
```